REFINEMENT VERSUS REFRESH

Compute Allocation for Closed-Loop Robot Policies

Alba María Téllez Fernández MAAT Institute
Machine Intelligence Summit | Zürich | 10 November 2026
alba.tellez@maatinstitute.com

A robot can spend compute improving its current action prediction—or use a newer observation to plan again. Which is worth the cost?

01 The question

Chunked vision–language–action policies generate a sequence of future actions. A robot executes only a prefix before asking the policy for a new chunk. Inference is costly, so two knobs compete for the same budget:

REFINE MORE
increase denoising steps d

same inference budget

REFRESH SOONER
reduce executed prefix h

Hypothesis to test
Does additional denoising improve closed-loop task success enough to justify its cost, or is computation better saved for observation-conditioned replanning?

02 What is measured

Let d denote denoising steps and h the number of actions executed from a predicted chunk before refreshing. For full execution prefixes, the nominal neural arithmetic rate is

$$b(h, d) = \frac{F_{\text{gen}}(d)}{h}$$

where $F_{\text{gen}}(d)$ is the reference-calibrated arithmetic for one full chunk generation.

82.1%
of native counted generation arithmetic is independent of the denoising-step count.
Reducing denoising alone leaves most of the cost of generating a new prediction intact.

03 Evaluation split

Discovery	12 configurations; four LIBERO Spatial tasks; 480 attempts
Confirmation	Seven frozen configurations; eight distinct Goal and Object tasks; 1,120 attempts

The eight confirmation tasks were excluded from configuration selection. The principal contrast uses 160 paired attempts per condition, matched by task, initial state and policy seed.

04 Main result: refine less

At a fixed two-action execution prefix ($h = 2$), compare two versus five denoising steps.



+10.6 pp
paired success difference, $d = 2$ minus $d = 5$
Descriptive, task-conditional 95% interval: +4.4 to +16.9 percentage points.

−5.9%
reference-calibrated neural arithmetic per control step when reducing d from 5 to 2 at $h = 2$.

05 Refresh less often?

Hold denoising fixed at $d = 2$. Compare executing two versus five actions before replanning.

Setting	Success	TFLOPs / attempt
$h = 2,\ d = 2$	139/160	41.1
$h = 5,\ d = 2$	138/160	16.1

−60.7%
reference-calibrated arithmetic per attempted episode for $h = 5$ versus $h = 2$, at $d = 2$.

Important: Similar observed success is *not* evidence of equivalent quality. The uncertainty interval does not establish non-inferiority or equivalence.

06 Compute is not latency

The reported resource axis counts reference-calibrated dense neural FLOPs. It does not measure physical-robot latency, asynchronous overlap, or energy.

07 Interpretation

- Additional denoising is **not uniformly beneficial** for closed-loop success on the evaluated tasks.
- The fixed cost of generating a new prediction limits savings from denoising reduction alone.
- Less frequent refresh can substantially reduce episode arithmetic, but its effect on quality remains uncertain.
- A simple cost-aware selection rule matched the fitted response model at three compute ceilings; fitting showed no demonstrated advantage.

08 What the study does not claim

- A universally optimal allocation strategy or an adaptive scheduler.
- Isolation of observation freshness: changing h also changes which predicted action positions are executed.
- Cross-policy generalization, real-time asynchronous gains, or physical-robot transfer.

09 Reproducibility

One frozen SmolVLA checkpoint; synchronous LIBERO simulation; separate discovery and confirmation tasks. The original study reports paired execution and internal action/outcome consistency checks. Independent model-free replay of the archived primary outcomes is a subsequent validation step, not evidence used for the results shown here.

Paper and code

github.com/amtellezfernandez/

refinement-refresh-robot-policies

Refinement versus Refresh: Compute Allocation for Closed-Loop Robot Policies. NeurIPS 2026 RoboPAD workshop.

THE DEPLOYMENT TRADE-OFF



TAKEAWAY
Inference efficiency is a closed-loop allocation problem.
Evaluate task success together with the cost and frequency of observation-conditioned replanning, rather than minimizing denoising steps in isolation.